

OPERATION LENS

v2

Evidence-Driven Intelligence Platform
Architecture & Build Document | 2026

Classification	OFFICIAL – DEMO ONLY
Version	2.0 – Evidence-Driven Architecture
Supersedes	Operation Lens v1.0
Status	Draft for Review
Environment	Local – Mac Mini (64 GB Unified Memory)
Key v2 Shifts	<i>DuckDB evidence store Entity normalisation Claim validation Structured relationship extraction</i>

1. Executive Summary

Operation Lens v2 is a fully local-first, evidence-driven intelligence analysis system for law enforcement PDF corpora. It supersedes v1 with a fundamental architectural shift: where v1 prioritised retrieval breadth and visual impact, v2 prioritises evidential rigour — every relationship asserted by the system is traceable to a specific span of text in a specific document on a specific page.

The system ingests thousands of PDF documents, extracts and normalises named entities, constructs a structured evidence graph, and answers investigative queries with span-level citations and confidence scores. The local-first design ensures no document content leaves the host machine under any operational configuration.

1.1 What Changed from v1

Dimension	v1	v2
Graph store	<i>NetworkX (in-memory Python)</i>	DuckDB (relational evidence store, persistent)
Entity deduplication	<i>None — same entity named differently = two nodes</i>	Normalisation + alias resolution across corpus
Relationship source	<i>Co-occurrence in same chunk</i>	Typed relationships with supporting text spans
Answer grounding	<i>Chunks passed to LLM, trust LLM</i>	Claim validation against evidence after generation
Retrieval modes	<i>Vector + graph traversal</i>	Exact match + keyword (FTS5) + vector + graph expansion
Cloud LLM use	<i>Query text sent to OpenRouter</i>	Only redacted structured evidence sent externally
Confidence model	<i>Not present</i>	Per-relationship and per-answer confidence scoring

1.2 Design Principles

- Evidence first: no claim is made without a traceable supporting text span
- Local by default: all inference runs on-device; cloud use is opt-in and data-safe
- Normalise aggressively: the same person under three different name spellings is one entity, not three
- Confidence is explicit: relationships and answers carry scores, not just text
- Auditable by design: every query, retrieval, and answer is logged for disclosure purposes

2. Architecture Overview

2.1 System Diagram



2.2 Component Summary

Component	Technology	Notes
PDF Extraction	pdfplumber + Tesseract	Text-native + OCR fallback
General NER	GLiNER large-v2.1	Zero-shot, 8 entity types
Rule-Based NER	regex + spaCy patterns	Plates, phones, case refs, postcodes

Entity Normaliser	Custom Python + fuzzy match	Alias resolution, deduplication
Relationship Extractor	Pattern rules + Llama 3.1 8B	Typed edges with text spans
Embedding Model	nomic-embed-text via Ollama	Local, 384-dim, no external calls
Vector Store	LanceDB	Serverless, Arrow-native
Evidence Store	DuckDB	Replaces NetworkX; persistent, queryable
Full-Text Search	DuckDB FTS5	Keyword search over chunk text
LLM — Reasoning	Llama 3.1 70B Q4_K_M (Ollama)	~40 GB, local, default inference
LLM — Extraction	Llama 3.1 8B Q4 (Ollama)	Lightweight; relationship extraction only
LLM — Cloud	OpenRouter (redacted evidence)	Optional; no raw document text sent
API Backend	FastAPI	Async, full audit log
Frontend	Vanilla JS + D3.js	Terminal aesthetic, evidence panel

3. Data Model

v2 replaces the NetworkX in-memory graph with DuckDB — a persistent, SQL-queryable, embedded relational database. This is the most structurally significant change from v1. The entire evidence graph lives in DuckDB as typed tables, queryable with standard SQL and joinable across the full corpus without loading anything into memory.

3.1 Why DuckDB Over NetworkX

Persistence	DuckDB writes to disk. Restart the service and the graph is intact. NetworkX required re-ingestion or pickle files.
Queryability	SQL JOINS replace custom Python traversal. Cross-document queries that required manual graph walking are now single SQL statements.
Scale	DuckDB handles hundreds of millions of rows on a laptop. NetworkX degrades as graph complexity grows.
Evidence linkage	Every relationship row holds a foreign key to the exact chunk and text span that supports it. Not possible with graph nodes alone.
FTS5 integration	DuckDB's full-text search extension gives keyword search over chunk text at no additional infrastructure cost.

3.2 Core Schema

3.2.1 documents

```
CREATE TABLE documents (
  doc_id      TEXT PRIMARY KEY,
  filename    TEXT NOT NULL,
  filepath    TEXT NOT NULL,
  classification TEXT DEFAULT 'OFFICIAL',
  page_count  INTEGER,
  ingested_at TIMESTAMP DEFAULT now(),
  ocr_used    BOOLEAN DEFAULT false
);
```

3.2.2 chunks

```
CREATE TABLE chunks (
  chunk_id    TEXT PRIMARY KEY, -- uuid
  doc_id      TEXT REFERENCES documents(doc_id),
  page        INTEGER NOT NULL,
  chunk_index INTEGER NOT NULL,
  text        TEXT NOT NULL,
  token_count INTEGER,
  ingested_at TIMESTAMP DEFAULT now()
);

-- Full-text search index over chunk text
PRAGMA create_fts_index('chunks', 'chunk_id', 'text');
```

3.2.3 entities + entity_aliases

```
CREATE TABLE entities (
  entity_id    TEXT PRIMARY KEY, -- uuid
  canonical_name TEXT NOT NULL, -- normalised form
  entity_type  TEXT NOT NULL, -- PERSON / LOCATION / VEHICLE etc.
  first_seen_doc TEXT REFERENCES documents(doc_id),
  mention_count INTEGER DEFAULT 1,
  confidence   FLOAT DEFAULT 1.0
);

CREATE TABLE entity_aliases (
```

```

alias_id      TEXT PRIMARY KEY,
entity_id     TEXT REFERENCES entities(entity_id),
alias_text    TEXT NOT NULL,          -- raw surface form
source_doc    TEXT REFERENCES documents(doc_id),
source_chunk  TEXT REFERENCES chunks(chunk_id)
);

```

3.2.4 relationships + relationship_evidence

```

CREATE TABLE relationships (
  rel_id      TEXT PRIMARY KEY,
  source_entity TEXT REFERENCES entities(entity_id),
  target_entity TEXT REFERENCES entities(entity_id),
  relation_type TEXT NOT NULL,
  -- Types: MENTIONED_WITH | OBSERVED_AT | ASSOCIATED_WITH |
  --         LINKED_TO | INFERRED_LINK
  confidence  FLOAT NOT NULL,        -- 0.0 - 1.0
  first_evidenced TIMESTAMP
);

CREATE TABLE relationship_evidence (
  evidence_id TEXT PRIMARY KEY,
  rel_id      TEXT REFERENCES relationships(rel_id),
  chunk_id    TEXT REFERENCES chunks(chunk_id),
  doc_id      TEXT REFERENCES documents(doc_id),
  page        INTEGER NOT NULL,
  span_start  INTEGER,               -- character offset in chunk
  span_end    INTEGER,
  span_text   TEXT NOT NULL,         -- the exact supporting passage
  extraction_method TEXT             -- 'pattern' | 'llm' | 'co-occurrence'
);

```

3.2.5 Audit tables

```

CREATE TABLE queries (
  query_id     TEXT PRIMARY KEY,
  query_text   TEXT NOT NULL,
  intent       TEXT,
  llm_backend  TEXT,                -- 'local' | 'openrouter'
  submitted_at TIMESTAMP DEFAULT now()
);

CREATE TABLE answer_spans (
  span_id      TEXT PRIMARY KEY,
  query_id     TEXT REFERENCES queries(query_id),
  claim_text   TEXT NOT NULL,
  supporting_evidence TEXT[],        -- array of evidence_ids
  confidence   FLOAT,
  validated    BOOLEAN              -- result of claim validator
);

```

4. Entity Normalisation and Alias Resolution

This is the capability v1 lacked entirely and which most RAG systems ignore. Without it, 'Marcus Webb', 'M. Webb', 'Webb, Marcus', and 'the suspect' are four separate nodes. Cross-document relationship discovery fails silently — the graph is wrong, not missing.

4.1 The Problem in Detail

Why normalisation matters more than retrieval

Consider a corpus where Document A refers to 'Marcus Webb', Document B refers to 'M. Webb (suspect)', and Document C refers only to 'Webb'. A naive system creates three entity nodes. A query for 'all locations associated with Marcus Webb' misses everything in Documents B and C. The system appears to answer but returns an incomplete — potentially misleading — operational picture.

4.2 Normalisation Pipeline

```
Raw surface form (from GLiNER or rule extractor)
↓
Step 1: Type-specific normalisation
PERSON → title strip, name reorder (Webb Marcus → Marcus Webb)
VEHICLE → plate normalisation (rx71kld → RX71 KLD)
LOCATION → postcode normalisation, abbreviation expansion
PHONE → E.164 format (+44 7xxx xxxxxx)
↓
Step 2: Candidate lookup
Query entity table for same type, similar canonical name
Similarity: Jaro-Winkler distance (threshold: 0.88)
├── Match found → link as alias to existing entity
└── No match → create new canonical entity
↓
Step 3: Alias registration
Insert into entity_aliases (alias_text, entity_id, source_doc, source_chunk)
↓
Step 4: Mention count update
Increment entities.mention count
Update first seen doc if earlier document
```

4.3 Type-Specific Rules

Entity Type	Normalisation Rules	Example
PERSON	Strip titles (Mr/Mrs/DC/DCI/Sgt). Reorder surname-first forms. Lowercase then title-case.	'DC Webb, M.' → 'Marcus Webb'
VEHICLE	Remove spaces and punctuation. Uppercase. Re-insert standard spacing (AA00 AAA format).	'rx71kld' → 'RX71 KLD'
LOCATION	Expand common abbreviations (Rd → Road). Normalise postcode format. Strip floor/unit prefixes.	'14 Arkwright Rd, CH1' → '14 Arkwright Road, CH1'
PHONE	Strip formatting. Apply E.164. UK numbers: +44 prefix, remove leading 0.	'07712 345678' → '+447712345678'
CASE_REF	Uppercase. Remove spaces. Validate against known format patterns (URN, op name).	'op nightfall' → 'OP_NIGHTFALL'

ORGANISATION	Strip legal suffixes (Ltd, plc). Lowercase then title-case. Known alias table.	'Arkwright Holdings Ltd' → 'Arkwright Holdings'
--------------	--	---

4.4 Alias Resolution — Cross-Document Example

```
Document NF-INT-001: 'Marcus Webb'           → entity id: ENT-001 (canonical)
Document NF-INT-002: 'M. Webb (suspect)'      → alias → ENT-001
Document NF-SURV-003: 'Webb'                  → alias → ENT-001 (type match + Jaro-
Winkler)
Document NF-TEC-008: 'the suspect'            → NOT resolved (pronoun — future
coreference work)
```

```
Query: 'all locations associated with Marcus Webb'
→ Resolves to ENT-001
→ Joins relationships WHERE source entity = 'ENT-001' OR target entity = 'ENT-001'
→ Returns evidence from NF-INT-001, NF-INT-002, and NF-SURV-003
→ v1 would have returned evidence from NF-INT-001 only
```

5. Structured Relationship Extraction

v1 inferred relationships from entity co-occurrence within a chunk — if two entities appeared in the same 400-token window, they got an edge. This produces a dense but low-confidence graph where every entity is connected to every other entity it shares a paragraph with, regardless of whether the text actually asserts a relationship.

v2 extracts typed relationships with explicit supporting evidence. Every edge is a claim backed by a specific text span.

5.1 Relationship Types

MENTIONED_WITH	Two entities appear in the same document passage. Lowest confidence. No directional assertion. Generated by co-occurrence fallback only.
OBSERVED_AT	A person or vehicle is described as being physically present at a location at a specific time. Generated by pattern extractor or LLM.
ASSOCIATED_WITH	A person is described as connected to another person or organisation without a specific observed event. Generated by pattern extractor or LLM.
LINKED_TO	A financial, communications, or indirect forensic link is established. Generated by LLM-assisted extraction.
INFERRED_LINK	Multi-hop synthesis: $A \rightarrow B \rightarrow C$ therefore $A - C$. Highest-level, lowest confidence. Generated only in reasoning mode.

5.2 Two-Stage Extraction Pipeline

Stage 1: Pattern-Based Extraction

Fast, deterministic, high precision on well-formed text. Runs on every chunk at ingestion time.

```
# Patterns (simplified)
OBSERVED_AT:
  '{PERSON} was observed at {LOCATION}'
  '{VEHICLE} was seen at {LOCATION} at {DATETIME}'
  '{PERSON} attended {LOCATION} on {DATETIME}'

ASSOCIATED_WITH:
  '{PERSON} is known to associate with {PERSON}'
  '{PERSON} is believed to be connected to {ORGANISATION}'
  '{PERSON} is a member of {ORGANISATION}'

LINKED_TO:
  '{PERSON} 's phone ({PHONE}) was in contact with'
  'account {FINANCIAL} was used to pay {PERSON / ORGANISATION}'
```

Stage 2: LLM-Assisted Extraction

Runs on chunks where pattern extraction yielded no typed relationships, or where entity density is high. Uses Llama 3.1 8B — not the 70B — to keep extraction fast and cost-free.

```
SYSTEM:
Extract typed relationships from the following intelligence text.
Return JSON only. Schema:
{ relationships: [{ source, target, type, span, confidence }] }
Types: OBSERVED_AT | ASSOCIATED_WITH | LINKED_TO | MENTIONED_WITH
Use canonical entity names only. If no relationship is clearly stated, return [].

TEXT: {chunk_text}
ENTITIES_PRESENT: {entity_list_from_gliner}
```

The 8B model is used for extraction, not reasoning. It is fast enough to run across thousands of chunks without becoming a bottleneck, and small enough to leave headroom for the 70B model to handle query-time reasoning in parallel.

5.3 Evidence Linkage

Every extracted relationship is stored in relationships and backed by one or more rows in relationship_evidence. The evidence row contains:

- chunk_id and doc_id — which document and chunk
- page — page number for disclosure/citation
- span_start and span_end — character offsets into the chunk text
- span_text — the exact passage supporting the relationship
- extraction_method — 'pattern', 'llm', or 'co-occurrence'

Confidence scoring

Pattern-extracted relationships: 0.85 default. LLM-extracted: LLM-reported confidence (0.0–1.0, clamped). Co-occurrence fallback: 0.35. Confidence is stored on the relationship row and surfaced in query responses alongside citations.

6. Query Pipeline

6.1 Query Parsing

Before retrieval, the query is parsed to identify intent and entities. This determines which retrieval modes to activate and how to weight results.

```
Input: 'What locations is Marcus Webb connected to across all documents?'

→ Intent:    entity relationship query
→ Entities:  ['Marcus Webb' (PERSON)]
→ Normalise: 'Marcus Webb' → ENT-001
→ Retrieval plan:
    - Graph expansion (primary): relationships WHERE entity = ENT-001
    - Vector search (secondary): embed query, top-K chunks
    - FTS5 (tertiary): keyword 'Marcus Webb' OR known aliases
    - Exact match: skip (no case ref or plate in query)
```

6.2 Four Retrieval Modes

6.2.1 Exact Match

Activated when query contains a structured identifier: case reference, vehicle plate, phone number, or financial reference. Direct DuckDB lookup against `entity_aliases.alias_text` or `entities.canonical_name`.

6.2.2 Keyword Search (FTS5)

DuckDB's full-text search extension indexes all chunk text. Returns chunks containing the query terms or known aliases of matched entities. Handles spelling variations and partial names better than vector search for proper nouns.

```
SELECT c.chunk_id, c.doc_id, c.page, c.text,
       fts_main_chunks.match_bm25(chunk_id, ?) AS score
FROM chunks c
JOIN fts_main_chunks ON c.chunk_id = fts_main_chunks.chunk_id
WHERE score IS NOT NULL
ORDER BY score DESC LIMIT 20;
```

6.2.3 Vector Search

Query is embedded with `nomic-embed-text`. LanceDB returns top-K chunks by cosine similarity. Best for conceptual and contextual queries where exact terms are not known ('what financial arrangements were discovered').

6.2.4 Graph Expansion

For entity queries, the normalised entity IDs are used to traverse the DuckDB relationship graph. First and second-degree neighbours are returned with their evidence. This is now a SQL JOIN rather than Python graph traversal:

```
-- First-degree: all entities directly related to ENT-001
SELECT r.*, re.span_text, re.doc_id, re.page
FROM relationships r
JOIN relationship_evidence re ON r.rel_id = re.rel_id
WHERE r.source_entity = 'ENT-001'
      OR r.target_entity = 'ENT-001'
ORDER BY r.confidence DESC;

-- Second-degree: entities connected to first-degree neighbours
-- (CTE recursive query or two-step JOIN)
```

6.3 Merge and Rerank

Results from all four modes are merged into a unified evidence pool. Deduplication is by `chunk_id`. Reranking applies the following weights:

Cross-document relationship	Weight: 1.5 ×. A relationship evidenced in two or more documents is worth more than one from a single document.
Graph expansion result	Weight: 1.3 ×. Structured relationship evidence is preferred over unstructured chunk retrieval.
Pattern-extracted relationship	Weight: 1.2 ×. Higher confidence than LLM-extracted.
Vector search result	Weight: 1.0 ×. Baseline. Good for context; not for assertions.
FTS5 keyword result	Weight: 0.9 ×. Useful for surfacing overlooked chunks; lower precision.

6.4 Evidence Packet Construction

The merged, reranked results are assembled into a structured evidence packet — the input to LLM synthesis. The packet is not a raw dump of chunk text; it is a structured object:

```
evidence_packet = {
  query_id: '...',
  entities_resolved: [
    { canonical: 'Marcus Webb', entity_id: 'ENT-001', aliases: [...] }
  ],
  relationships: [
    {
      source: 'Marcus Webb', target: '14 Arkwright Road',
      type: 'OBSERVED_AT', confidence: 0.91,
      evidence: [
        { doc: 'NF-SURV-003', page: 3,
          span: 'Webb was observed at 14 Arkwright Road at 21:40 on...' }
      ]
    },
    ...
  ],
  chunks: [ { doc_id, page, text } ... ] // supporting context
}
```

7. Claim Validation

This is the capability that most directly distinguishes v2 from standard RAG. After the LLM generates an answer, each factual claim in the answer is validated against the evidence packet before the response is returned to the user. Hallucinated or unsupported claims are flagged rather than presented as fact.

7.1 Why Claim Validation Matters for Law Enforcement

The hallucination risk in evidential contexts

A law enforcement analyst acting on an AI-generated intelligence summary may make operational decisions — resourcing, surveillance, arrest — based on that summary. A hallucinated relationship ('Webb was also observed at the depot on 14 March') that sounds plausible but is not in the documents is not just incorrect: it is potentially a disclosure failure, an investigative dead end, and a risk to the integrity of proceedings. Claim validation does not eliminate this risk but it surfaces it explicitly.

7.2 Validation Process

```
LLM Answer:
'Marcus Webb was observed at 14 Arkwright Road on the evening
of 7th November [NF-SURV-003, p.3]. He is also associated
with Rania Khalil through the depot location [NF-SURV-004, p.2,
NF-FIN-005, p.7].'
```

```
Claim Validator:
1. Extract discrete claims from answer text
2. For each claim:
  a. Identify cited evidence IDs or doc/page references
  b. Retrieve span text from relationship evidence for cited evidence
  c. Check: does the span text support the claim semantically?
    → Use small LLM (8B) for entailment check: does span ⊨ claim?
  d. Score: SUPPORTED | PARTIALLY_SUPPORTED | UNSUPPORTED
3. Flag UNSUPPORTED claims in response
4. Store validation results in answer spans table
```

7.3 Response Structure

```
{
  "answer": "Marcus Webb was observed at 14 Arkwright Road...",
  "claims": [
    {
      "text": "Webb was observed at 14 Arkwright Road on 7th November",
      "status": "SUPPORTED",
      "confidence": 0.91,
      "citations": [{ "doc": "NF-SURV-003", "page": 3,
        "span": "Webb was observed at 14 Arkwright..." }]
    },
    {
      "text": "He is associated with Rania Khalil through the depot",
      "status": "PARTIALLY_SUPPORTED",
      "confidence": 0.64,
      "citations": [{ "doc": "NF-SURV-004", "page": 2 },
        { "doc": "NF-FIN-005", "page": 7 } ],
      "note": "Indirect association via shared location. No direct link stated."
    }
  ]
}
```

PARTIALLY_SUPPORTED and UNSUPPORTED claims are displayed with an amber or red indicator in the UI. The analyst sees the answer and its confidence posture simultaneously — not just a confident-sounding paragraph.

8. LLM Strategy

v2 uses three models with distinct, non-overlapping roles. The 70B is not used for everything — that was a v1 simplification that made the system slow to respond and wasteful of compute.

8.1 Model Assignments

Model	Task	Why This Model	Approx Speed
Llama 3.1 70B Q4 (Ollama)	Query-time reasoning, multi-hop synthesis, report generation	Highest local reasoning quality. Used only where quality matters.	8–15 tok/s
Llama 3.1 8B Q4 (Ollama)	Relationship extraction, entity cleanup, claim entailment check	Fast and cheap. Runs in parallel with ingestion. Low latency.	60–80 tok/s
OpenRouter (cloud)	Complex multi-document synthesis when explicitly enabled	Cloud reasoning on redacted evidence only. No raw text sent.	Variable

8.2 OpenRouter Data Safety

When OpenRouter is enabled, the system does not send document text. It sends only the structured evidence packet — canonical entity names, relationship types, confidence scores, and page references. Raw text spans are redacted before transmission:

```
Sent to OpenRouter:
{
  "entities": ["Marcus Webb (PERSON)", "14 Arkwright Road (LOCATION)"],
  "relationships": [
    { source: 'Marcus Webb', target: '14 Arkwright Road',
      type: 'OBSERVED_AT', confidence: 0.91,
      evidence count: 2, docs: ['NF-SURV-003', 'NF-FIN-005'] }
  ],
  "query": "What is the connection between Webb and Khalil?"
}

NOT sent:
- Any raw chunk text
- Any span_text from relationship_evidence
- Any witness statement or interview content
```

This design makes the data sovereignty position defensible even when OpenRouter is used: what leaves the machine is a structured summary, not document content. The UI makes this explicit with a banner when cloud mode is active.

9. Build Guide

9.1 Prerequisites

Host Machine	Mac Mini, 64 GB unified memory
Python	3.11+ via pyenv
Ollama	Latest — pull llama3.1:70b-instruct-q4_K_M and llama3.1:8b-instruct-q4_K_M and nomic-embed-text
Docker Desktop	For containerised FastAPI service (optional for local dev)
Tesseract OCR	brew install tesseract tesseract-lang
DuckDB	pip install duckdb (embedded, no server needed)

9.2 Ollama Setup

```
ollama pull llama3.1:70b-instruct-q4_K_M # ~40 GB
ollama pull llama3.1:8b-instruct-q4_K_M # ~5 GB
ollama pull nomic-embed-text           # ~500 MB

ollama list # verify all three present

# Start Ollama server (if not running as service)
ollama serve
```

9.3 Python Dependencies

```
pip install \
  pdfplumber pytesseract pillow \
  gliner \
  duckdb \
  lancedb pyarrow \
  rapidfuzz \           # Jaro-Winkler for alias resolution
  spacy \                # rule-based NER patterns
  httpx \
  fastapi uvicorn \
  openai                 # OpenRouter uses OpenAI-compatible API

python -m spacy download en_core_web_sm
```

9.4 Project Structure

```
operation-lens-v2/
├── ingestion/
│   ├── extractor.py           # pdfplumber + Tesseract
│   ├── chunker.py            # semantic chunker, 256-400 tok
│   ├── ner_gliner.py          # GLiNER general NER
│   ├── ner_rules.py           # regex: plates, phones, case refs
│   ├── normaliser.py          # entity normalisation + alias resolution
│   ├── relationship_extractor.py # pattern + 8B LLM
│   ├── embedder.py            # nomic-embed-text via Ollama
│   ├── duck_store.py          # DuckDB schema + write operations
│   └── vector_store.py         # LanceDB write operations
├── query/
│   ├── parser.py              # intent + entity detection
│   ├── retriever_exact.py      # DuckDB exact match
│   ├── retriever_fts.py        # DuckDB FTS5 keyword search
│   ├── retriever_vector.py     # LanceDB vector search
│   ├── retriever_graph.py      # DuckDB relationship JOIN
│   ├── reranker.py             # merge, dedup, weight
│   ├── evidence_builder.py     # assemble evidence packet
│   ├── llm_router.py           # 70B local / OpenRouter toggle
│   └── claim_validator.py      # post-generation validation
```

```

├── api/
│   ├── main.py
│   └── routes/
│       ├── ingest.py
│       ├── query.py
│       ├── graph.py
│       └── audit.py
├── frontend/
│   ├── index.html
│   ├── evidence_panel.js    # span-level citation display
│   ├── graph.js             # D3.js entity graph
│   └── confidence_badge.js   # SUPPORTED / PARTIAL / UNSUPPORTED
├── data/
│   ├── pdfs/
│   ├── evidence.duckdb      # single file - entire evidence store
│   └── lancedb/
└── docker-compose.yml

```

9.5 Key Code Patterns

9.5.1 DuckDB Initialisation

```

# duck_store.py
import duckdb

def init_db(path: str = 'data/evidence.duckdb') -> duckdb.DuckDBPyConnection:
    con = duckdb.connect(path)
    con.execute('INSTALL fts; LOAD fts;')
    con.execute(SCHEMA SQL)    # CREATE TABLE statements
    con.execute(
        "PRAGMA create_fts_index('chunks', 'chunk_id', 'text');"
    )
    return con

```

9.5.2 Entity Normaliser

```

# normaliser.py
from rapidfuzz.distance import JaroWinkler

THRESHOLD = 0.88

def resolve_entity(surface: str, etype: str, con) -> str:
    canonical = normalise(surface, etype)
    candidates = con.execute(
        'SELECT entity_id, canonical_name FROM entities WHERE entity_type = ?',
        [etype]
    ).fetchall()
    for eid, cname in candidates:
        if JaroWinkler.similarity(canonical, cname) >= THRESHOLD:
            register_alias(surface, eid, con)
            return eid
    return create_entity(canonical, etype, con)

```

9.5.3 Claim Validator

```

# claim_validator.py
async def validate_claim(claim: str, evidence_spans: list[str]) -> dict:
    prompt = f'''
Does the following evidence support this claim?
Claim: {claim}
Evidence: {chr(10).join(evidence_spans)}
Reply with JSON: {{ supported: true|false, partial: true|false,
                    confidence: 0.0-1.0, note: str }}
'''
    result = await ollama_8b(prompt)
    return parse_json(result)

```

10. Build Timeline

v2 is more complex than v1. The additional components — normaliser, relationship extractor, DuckDB schema, claim validator — add approximately two days to the v1 timeline.

Day	Tasks	Deliverable	Hours
1	Ollama setup, PDF extractor, Tesseract fallback, semantic chunker	extractor.py + chunker.py	5–6
2	GLiNER NER, rule-based NER (plates/phones/refs), DuckDB schema, FTS5 init	ner + duck_store.py working	5–6
3	Entity normaliser, Jaro-Winkler alias resolution, alias table population	normaliser.py, cross-doc dedup live	5–6
4	Relationship extractor: pattern rules + 8B LLM extraction, evidence linkage to DuckDB	relationship_extractor.py working	5–6
5	LanceDB embeddings, all four retrieval modes, merge/rerank, evidence packet builder	Full retrieval pipeline	5–6
6	70B LLM synthesis, claim validator, OpenRouter redacted mode, audit logging	Query pipeline end-to-end	5–6
7	FastAPI routes, frontend: evidence panel, confidence badges, D3 entity graph	Working demo UI	5–6
8	Synthetic corpus (Operation Nightfall), demo script, polish, README, LinkedIn draft	Demo-ready + blog draft	4–5

Total: approximately 7–8 days for a full, polished v2 demo. The normaliser (Day 3) and claim validator (Day 6) are the highest-risk components — allocate buffer here if compressing the timeline.

11. Data Sovereignty and Security Posture

11.1 Data Flow by Component

Component	Data Leaves Machine?	Notes
PDF extraction	Never	pdfplumber and Tesseract run in-process
GLiNER NER	Never	Model weights local; all inference on-device
Rule-based NER	Never	Pure Python regex patterns
Entity normalisation	Never	Jaro-Winkler, DuckDB queries, all local
Relationship extraction (pattern)	Never	Deterministic pattern matching
Relationship extraction (8B LLM)	Never	Llama 3.1 8B via local Ollama
Embedding (nomic)	Never	Ollama local inference
LanceDB	Never	File-system native, no network socket
DuckDB	Never	Embedded database, single file on disk
70B LLM synthesis	Never	Llama 3.1 70B via local Ollama
Claim validation	Never	8B LLM local entailment check
OpenRouter (if enabled)	Structured evidence only	Redacted: no raw text, no span content. Explicit UI banner. Every call logged in audit table.

12. Known Limitations and Future Work

12.1 v2 Limitations

- No coreference resolution: pronouns and definite references ('the suspect', 'the vehicle') are not resolved to canonical entities. This is the single biggest accuracy gap remaining.
- Alias resolution is fuzzy but not semantic: 'the Webb associate' will not resolve to Marcus Webb via Jaro-Winkler alone. Coreference or LLM-based resolution would handle this.
- Relationship extraction on degraded OCR text is unreliable: the pattern extractor requires well-formed sentences. Very poor OCR output (scanning artefacts, skewed text) reduces extraction recall.
- Claim validator latency: an additional 8B LLM call per claim adds ~2–4 seconds per answer. Acceptable for a demo; a production system would batch validate or run validator asynchronously.
- DuckDB is single-writer: concurrent ingestion jobs must be serialised or use connection pooling. Not an issue at demo scale; relevant at production corpus size.

12.2 Production Path

- Coreference resolution: integrate coreferee (spaCy) or a fine-tuned NeuralCoref model. This would likely double the accuracy of pronoun-heavy intelligence documents.
- Neo4j: replace DuckDB graph tables with Neo4j for native Cypher traversal, APOC graph algorithms, and multi-hop path finding at scale.
- Classification-aware access control: tag documents and entities with classification levels; enforce at query time so analysts only see what their clearance permits.
- Async claim validation: decouple validation from the response path; return answer immediately with 'validation pending' status, push validated result when complete.
- Active learning for relationship extraction: log extraction failures and borderline cases; use them to fine-tune the 8B model on domain-specific relationship patterns.

Appendix A — v1 to v2 Decision Rationale

A summary of the four architectural shifts and why each was made.

Shift	v1 Limitation	v2 Decision
DuckDB replacing NetworkX	<i>In-memory graph lost on restart. No SQL query interface. Graph traversal required custom Python. Could not join relationships to evidence spans.</i>	DuckDB provides persistence, SQL JOIN queries, FTS5 full-text search, and foreign-key linkage between relationships and their supporting text spans — all in a single embedded file.
Entity normalisation + alias resolution	<i>Same entity under different name forms created multiple disconnected nodes. Cross-document relationship discovery silently incomplete. No deduplication.</i>	Jaro-Winkler similarity + type-specific normalisation rules resolve aliases at ingestion time. All surface forms link to a single canonical entity_id. Cross-document queries are complete.
Claim validation	<i>LLM answers trusted unconditionally. Hallucinated relationships presented alongside evidenced ones with no distinction. Significant risk in operational/legal context.</i>	Post-generation validation checks each claim against its cited evidence spans using the 8B model. SUPPORTED / PARTIALLY_SUPPORTED / UNSUPPORTED status surfaced explicitly in the UI.
Structured relationship extraction	<i>Co-occurrence in same chunk = relationship edge, regardless of whether the text asserts any relationship. Dense but low-precision graph.</i>	Two-stage extraction: deterministic pattern rules first, 8B LLM for residual chunks. Every edge stores the exact text span that supports it. Confidence scores reflect extraction method.

OFFICIAL – DEMO ONLY | All persons, operations and case references in this document are entirely fictitious.